Clase 2: Modelo Relacional y Álgebra Relacional

Hasta ahora

- Todo el mundo necesita manejar datos
- Salvo que queramos programar cosas que no tienen que ver con nuestro problema, conviene utilizar un DBMS
- Arquitectura de capas:
  - Usuarios finales ven el **modelo lógico**
  - El sistema ejecuta las acciones
  - El usuario no necesita saber *cómo* el sistema ejecuta las acciones
- Hoy vamos a formalizar el modelo que subyace a los DBMS relacionales como **PostgreSQL**

Objetivos de la semana

Al final de estas clases podrás:

1. **Definir** un esquema relacional (relaciones, atributos, dominios, llaves)
2. **Expresar** consultas en álgebra relacional usando los operadores fundamentales
3. **Componer** operadores para resolver consultas complejas
4. **Reconocer** las limitaciones del álgebra relacional y la necesidad de operadores como la diferencia

Modelo Relacional

Los datos se almacenan como tablas

Películas

ID	Nombre	Año	Categoría	Calificación
1	Interstellar	2014	SciFi	8.6
2	The Revenant	2015	Drama	8.1
3	Dune	2021	SciFi	8.0
4	The Theory of Everything	2014	Biografía	7.7
5	Oppenheimer	2023	Drama	8.5

Películas

ID	Nombre	Año	Categoría	Calificación
1	Interstellar	2014	SciFi	8.6
2	The Revenant	2015	Drama	8.1
3	Dune	2021	SciFi	8.0
4	The Theory of Everything	2014	Biografía	7.7
5	Oppenheimer	2023	Drama	8.5

Distinguimos:

- **Relación:** a cada tabla le llamamos relación
- **Atributos:** son las columnas de la relación
- **Tuplas:** son las filas de la relación

Películas

ID	Nombre	Año	Categoría	Calificación
1	Interstellar	2014	SciFi	8.6
2	The Revenant	2015	Drama	8.1
3	Dune	2021	SciFi	8.0
4	The Theory of Everything	2014	Biografía	7.7
5	Oppenheimer	2023	Drama	8.5

- La relación se llama **peliculas**
- Sus atributos son: `id`, `nombre`, `año`, `categoria`, `calificacion`

Esquema

Para denominar relaciones escribimos su nombre y luego sus atributos entre paréntesis:

```
Peliculas(id, nombre, año, categoria, calificacion)
```

Un **esquema** es un conjunto de relaciones con sus atributos:

```
Peliculas(id, nombre, año, categoria, calificacion, director)
Actores(id, nombre, edad)
Actuo_en(id_actor, id_pelicula)
```

Dominio

En la práctica, asumimos que cada atributo tiene un **dominio** (float, integer, string, date, ...)

```
Peliculas(id:int,
          nombre:string,
          año:int,
          categoria:string,
          calificacion:float,
          director:string)
```

Esquema vs. Instancia

Una **instancia** de un esquema es un conjunto de tuplas para cada relación del esquema.

Esto es un **esquema** (la estructura):

ID	Nombre	Año	Categoría	Calificación

Esto es una **instancia** (los datos):

ID	Nombre	Año	Categoría	Calificación
1	Interstellar	2014	SciFi	8.6
2	The Revenant	2015	Drama	8.1
3	Dune	2021	SciFi	8.0

Restricciones de Integridad

Restricciones de integridad

Son restricciones que imponemos a un esquema que **todas las instancias** deben satisfacer.

La restricción más importante son las **llaves**.

Llaves: Definiciones

- **Superllave (superkey):** un conjunto de atributos tal que no existen dos tuplas con los mismos valores en todos esos atributos
- **Llave candidata:** una superllave **minimal** (si quitamos cualquier atributo, deja de ser superllave)
- **Llave primaria:** la llave candidata que elegimos como identificador principal

Ejemplo: Llaves

ID	Nombre	Año	Categoría	Calificación
1	Interstellar	2014	SciFi	8.6
2	Batman	2005	Acción	8.3
3	Dune	2021	SciFi	8.0
4	The Theory of Everything	2014	Biografía	7.7
5	Batman	1995	Acción	5.4

Nota: Batman Begins (2005) y Batman Forever (1995)

¿Cuál es la llave?

- `{id}` es llave (cada id es único)
- `{nombre}` **no** es llave (Batman aparece dos veces)

Notación de llaves

Cuando escribimos las relaciones, **subrayamos** las llaves primarias:

```
Peliculas(id, nombre, año, categoria, calificacion, director)
Actores(id, nombre, edad)
Actuo_en(id_actor, id_pelicula)
```

Llaves foráneas (teaser)

Una **llave foránea** es un atributo (o conjunto de atributos) que referencia la llave primaria de otra relación.

```
actuo_en(id_actor, id_pelicula)
           │           │
           ▼           ▼
      actores.id   peliculas.id
```

Esto garantiza la **integridad de las tablas**: no podemos insertar en `actuo_en` un `id_actor` que no exista en `actores`.

> Veremos llaves foráneas en detalle más adelante. Por ahora, esta conexión entre tablas la tendremos en mente a la hora de cruzar tabas, lo que se conoce como **joins**.

Consultando Bases de Datos

Actores

id	nombre	edad
1	Leonardo DiCaprio	49
2	Matthew McConaughey	54
3	Timothée Chalamet	28
4	Jessica Chastain	47
5	Cillian Murphy	48

Actuo_en

id_actor	id_pelicula
1	2
2	1
4	1
3	3
1	6
5	5

Peliculas

id	nombre	año	categoria	calificacion	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Dune	2021	SciFi	8.0	D. Villeneuve
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Oppenheimer	2023	Drama	8.5	C. Nolan
6	Inception	2010	SciFi	8.8	C. Nolan

Ejemplo 1

Liste el nombre de todos los actores

nombre
Leonardo DiCaprio
Matthew McConaughey
Timothée Chalamet
Jessica Chastain
Cillian Murphy

Ejemplo 2

Liste el nombre y la calificación de todas las películas

nombre	calificacion
Interstellar	8.6
The Revenant	8.1
Dune	8.0
The Theory of Everything	7.7
Oppenheimer	8.5
Inception	8.8

Ejemplo 3

Liste el nombre y la calificación de todas las películas con calificación inferior a 8.5

nombre	calificacion
The Revenant	8.1
Dune	8.0
The Theory of Everything	7.7

Ejemplo 4

Liste todas las películas de Nolan

id	nombre	año	categoria	calificacion	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
5	Oppenheimer	2023	Drama	8.5	C. Nolan
6	Inception	2010	SciFi	8.8	C. Nolan

Ejemplo 5

Liste todos los id de los actores de la película "Interstellar"

id_actor
2
4

Ejemplo 6

Liste cada actor junto a todas las películas en las que ha actuado

nombre_actor	nombre_pelicula
Leonardo DiCaprio	The Revenant
Leonardo DiCaprio	Inception
Matthew McConaughey	Interstellar
Timothée Chalamet	Dune
Jessica Chastain	Interstellar
Cillian Murphy	Oppenheimer

Ejemplo 7

Liste todas las películas en que actúe Leonardo DiCaprio y que sean dirigidas por C. Nolan

nombre
Inception

Ejemplo 8

Liste todas las películas y la calificación en que actúe Leonardo DiCaprio o que sean dirigidas por C. Nolan

nombre	calificacion
Interstellar	8.6
The Revenant	8.1
Inception	8.8
Oppenheimer	8.5

Ejemplo 9

Liste el nombre de todos los actores y directores

nombre
Leonardo DiCaprio
Matthew McConaughey
Timothée Chalamet
Jessica Chastain
Cillian Murphy
C. Nolan
A. Iñárritu
D. Villeneuve

Ejemplo 10

Liste el nombre de todos los actores dirigidos por C. Nolan y por A. Iñárritu (ambos)

id	nombre	edad
1	Leonardo DiCaprio	49

¿Qué podemos concluir?

- Los resultados de las consultas **también son tablas**
- Parecen haber operaciones en común

¿Qué operaciones necesitamos?

Ejemplo	Operación necesaria
1, 2	**Seleccionar columnas** (proyección)
3, 4	**Filtrar filas** (selección)
5, 6	**Cruzar tablas** (join)
9	**Combinar resultados** (unión)
10	**Intersectar resultados** (intersección)

Estas operaciones forman el **álgebra relacional**.

Álgebra Relacional

¿Qué es el álgebra relacional?

- Un **lenguaje teórico** para consultar bases de datos relacionales
- Posee un conjunto de operadores que toman tablas como input y devuelven tablas como output (**propiedad de clausura**)
- Es el **fundamento teórico de SQL**: toda consulta SQL tiene una expresión equivalente en álgebra relacional
- Durante el curso veremos cómo este lenguaje forma los cimientos de todos los lenguajes de consulta

Resumen de operadores

Operador	Símbolo	Descripción
Proyección	π	Selecciona columnas
Selección	σ	Filtra filas según condición
Unión	$\cup$	Combina tuplas de dos relaciones
Diferencia	$-$	Tuplas en R_1 que no están en R_2
Producto cruz	$\times$	Producto cartesiano de dos relaciones
Renombre	ρ	Cambia nombres de atributos/relaciones
Join	$\bowtie$	Producto cruz + selección
Natural join	$\bowtie$	Join sobre atributos con mismo nombre (derivado)
Intersección	$\cap$	Tuplas comunes a ambas relaciones
División	$\div$	Cuantificación universal

Operadores Fundamentales

Proyección (π)

Sea R una relación, entonces $\pi_{a_1,\ldots,a_n}(R)$ es una nueva relación que contiene sólo los atributos $a_1, \ldots, a_n$ de R.

Ejemplo 1: Liste el nombre de todos los actores:

$$\pi_{\text{nombre}}(\text{Actores})$$

Ejemplo 2: Liste el nombre y la calificación de todas las películas:

$$\pi_{(\text{nombre,calificacion}}(\text{Peliculas})$$

Proyección: eliminación de duplicados

Las relaciones son **conjuntos** de tuplas, por lo que la proyección **elimina duplicados**.

Ejemplo: ¿Cuántas filas tiene $\pi_{\text{categorias}}(\text{Peliculas})$?

categoria
SciFi
Drama
Biografía

Solo 3 filas, no 6. Aunque "SciFi" aparece 3 veces y "Drama" 2 veces en `Peliculas`, cada valor aparece **una sola vez** en el resultado.

Selección (σ)

Sea R una relación, entonces $\sigma_{\text{condición}}(R)$ es una nueva relación que contiene sólo las tuplas (filas) que satisfacen la condición.

Las condiciones pueden usar: $<, \leq, \geq, >, =, \neq$

Y se pueden combinar con: $\wedge$ (and), $\vee$ (or)

Selección: Ejemplos

Ejemplo 1: Liste todas las películas de C. Nolan:

$$\sigma_{\text{director}=\text{"C. Nolan"}}(\text{Peliculas})$$

Ejemplo 2: Películas con calificación inferior a 8.5 dirigidas por J. Marsh:

$$\sigma_{\text{calificacion}<8.5\wedge\text{director}=\text{"J. Marsh"}}(\text{Peliculas})$$

Composición: Proyección + Selección

El output de un operador es el input de otro. Esto nos permite **componer** operaciones.

Ejemplo: Nombre y calificación de películas con calificación inferior a 8.5:

$$\pi_{\text{nombre,calificacion}}(\sigma_{\text{calificacion}<8.5}(\text{Peliculas}))$$

Composición: Proyección + Selección

El output de un operador es el input de otro. Esto nos permite **componer** operaciones.

Ejemplo: Nombre y calificación de películas con calificación inferior a 8.5:

$$\pi_{\text{nombre,calificacion}}(\sigma_{\text{calificacion}<8.5}(\text{Peliculas}))$$

Importante: el orden importa. Si proyectamos primero y eliminamos `calificacion`, ya no podemos filtrar por ella:

$$\sigma_{\text{calificacion}<8.5}(\pi_{\text{nombre}}(\text{Peliculas}))$$

```
← ERROR: calificacion ya no existe
```

Unión ($\cup$)

Sean R_1, R_2 relaciones con la **misma cantidad de atributos y del mismo tipo**, entonces $R_1 \cup R_2$ es una nueva relación que contiene la unión de las tuplas de R_1 y R_2.

Ejemplo: Liste el nombre de todos los actores y directores:

$$\pi_{\text{nombre}}(\text{Actores}) \cup \pi_{\text{director}}(\text{Peliculas})$$

Observación: las relaciones son **conjuntos**, por lo que si una tupla está en R_1 y R_2, la unión la contiene **una sola vez**.

Renombre (ρ)

Renombrando atributos

Para cambiar nombres de atributos en una relación usamos el operador ρ:

$$\rho_{(\text{nombre}\rightarrow\text{name},\text{año}\rightarrow\text{year},\text{categoria}\rightarrow\text{category},\text{calificacion}\rightarrow\text{rating})}(\text{Peliculas})$$

Renombrando relaciones

Podemos guardar resultados intermedios con un nombre:

$$\rho_{\text{Actores_jovenes}}(\sigma_{\text{edad}<30}(\text{Actores}))$$

Luego podemos consultar:

$$\pi_{\text{nombre}}(\text{Actores_jovenes})$$

Producto Cruz ($\times$)

El operador $\times$ permite hacer el **producto cartesiano** de dos relaciones.

Si R tiene n tuplas y S tiene m tuplas, entonces $R \times S$ tiene $n \cdot m$ tuplas.

> El producto cruz combina **cada** tupla de R con **cada** tupla de S.

Producto Cruz: Ejemplo paso a paso

Consulta: Liste todos los id de los actores de la película "Interstellar"

Paso 1: Hacemos el producto cruz de peliculas y actuo_en:

`peliculas × actuo_en`

Peliculas.id	Peliculas.nombre	...	Actuo_en.id_actor	Actuo_en.id_pelicula
1	Interstellar	...	1	2
1	Interstellar	...	2	1
1	Interstellar	...	4	1
1	Interstellar	...	3	3
2	The Revenant	...	1	2
2	The Revenant	...	2	1
...	...	...	...	...

Esto genera **6 × 6 = 36 filas** (¡la mayoría no tienen sentido!)

Producto Cruz: Paso 2

Paso 2: Filtramos cuando `peliculas.id = actuo_en.id_pelicula`:

$$\sigma_{\text{peliculas.id}=\text{Actuo_en.id_pelicula}}(\text{Peliculas} \times \text{Actuo_en})$$

Peliculas.id	Peliculas.nombre	...	Actuo_en.id_actor	Actuo_en.id_pelicula
1	Interstellar	...	2	1
1	Interstellar	...	4	1
2	The Revenant	...	1	2
3	Dune	...	3	3
5	Oppenheimer	...	5	5
6	Inception	...	1	6

Ahora solo quedan las filas donde la relación tiene sentido.

Producto Cruz: Paso 3

Paso 3: Filtramos por "Interstellar" y proyectamos:

$$\pi_{\text{Actuo_en.id_actor}}(\sigma_{\text{Peliculas.id}=1}(\sigma_{\text{peliculas.id}=\text{Actuo_en.id_pelicula}}(\text{Peliculas} \times \text{Actuo_en})))$$

id_actor
2
4

¿Mas conveniente: el Join?

El producto cruz genera **|R| × |S|** tuplas — una explosión de cardinalidad.

En la práctica, casi siempre usamos producto cruz **seguido de selección** para filtrar las combinaciones que tienen sentido.

Esta combinación es tan común que tiene su propio operador: el **Join**.

Join

Join ($\bowtie$)

El **join** combina producto cruz + selección en un solo operador:

$$R \bowtie_{\text{condición}} S = \sigma_{\text{condición}}(R \times S)$$

Ejemplo: Liste los actores de "Interstellar":

$$\pi_{\text{Actuo_en.id_actor}}(\sigma_{\text{Peliculas.id}=1}(\text{Peliculas} \bowtie_{\text{peliculas.id}=\text{actuo_en.id_pelicula}} Actuo_en))$$

Joins Múltiples

Consulta: Liste cada actor junto a todas las películas en las que ha actuado.

Necesitamos cruzar **tres** tablas: actores, actuo_en, peliculas.

$$\pi_{\text{Actores.nombre, Peliculas.nombre}} (\text{Actores} \bowtie_{\text{Actores.id=Actuo_en.id_actor}} \text{Actuo_en}(\bowtie_{\text{Actuo_en.id_pelicula=Peliculas.id}} \text{Peliculas}))$$

Actores.nombre	Peliculas.nombre
Leonardo DiCaprio	The Revenant
Leonardo DiCaprio	Inception
Matthew McConaughey	Interstellar
Timothée Chalamet	Dune
Jessica Chastain	Interstellar
Cillian Murphy	Oppenheimer

Equijoin

Un **equijoin** es un join donde la condición es una **igualdad** entre atributos:

$$R \bowtie_{R.a=S.b} S$$

La mayoría de los joins en la práctica son equijoins.

Natural Join

Cuando los atributos en ambas relaciones tienen el **mismo nombre**, es posible no indicar la condición:

Si $R(a, \mathbf{b})$ y $S(\mathbf{b}, c)$:

$$R \bowtie S = \pi_{a,b,c}(\sigma_{R.b=S.b}(R \times S))$$

El natural join:

- Hace join automáticamente sobre **todos** los atributos con el mismo nombre
- Elimina las columnas duplicadas del resultado

Consultas monótonas y la diferencia

Un problema

Consulta: Liste los actores dirigidos por C. Nolan y **no** por A. Iñárritu.

¿Podemos resolver esto con los operadores que conocemos?
$(\pi, \sigma, \cup, \times, \bowtie)$

Un problema

Consulta: Liste los actores dirigidos por C. Nolan y **no** por A. Iñárritu.

¿Podemos resolver esto con los operadores que conocemos? ($\pi, \sigma, \cup, \times, \bowtie$)

¡No podemos! Necesitamos un nuevo concepto para entender por qué.

Consultas monótonas

Sea Q una expresión de álgebra relacional sobre un esquema S.

Q es **monótona** si para toda instancia I_1, I_2 sobre S:

> Si $I_1 \subseteq I_2$, entonces $Q(I_1) \subseteq Q(I_2)$

Consultas monótonas: intuición

Intuición: si agrego más datos a la base de datos, el resultado de la consulta **no disminuye**.

Ejemplo monótono: $\pi_{\text{nombre}}(Actores)$

- Si agrego un actor nuevo, el resultado crece (o se mantiene)
- Nunca puede disminuir al agregar datos

Ejemplo NO monótono: "actores que no han actuado en películas de Nolan"

- Si agrego una nueva tupla en `Actuo_en` que conecta a un actor con una película de Nolan, ese actor **desaparece** del resultado
- El resultado **disminuye** al agregar datos

Teorema de monotonía

Teorema: Toda consulta usando solamente los operadores ($\pi, \sigma, \cup, \times, \bowtie$) es monótona.

Argumento:

- Cada operador individualmente es monótono (se puede verificar caso por caso)
- La composición de funciones monótonas es monótona

Consecuencia: Con solo ($\pi, \sigma, \cup, \times, \bowtie$) **NO podemos** expresar consultas que digan "encuentra X que NO está en Y."

Para eso necesitamos el operador de **diferencia**.

Diferencia (–)

Sean R_1, R_2 relaciones con los mismos atributos, su diferencia $R_1 - R_2$ es una nueva relación que contiene las tuplas de R_1 que **no están** en R_2.

Diferencia: Ejemplo

Consulta: Liste los actores dirigidos por C. Nolan y **no** por A. Iñárritu.

Paso 1: actores dirigidos por C. Nolan:

$$\rho_{\text{Nolan}}\left(\pi_{\text{nombre}}\left(\text{Actores} \bowtie \text{Actuo_en} \bowtie \sigma_{\text{director}=\text{"C. Nolan"}}(\text{Peliculas})\right)\right)$$

Paso 2: actores dirigidos por A. Iñárritu:

$$\rho_{\text{Inarritu}}\left(\pi_{\text{nombre}}\left(\text{actores} \bowtie \text{Actuo_en} \bowtie \sigma_{\text{director}=\text{"A. Iñárritu"}}(\text{Peliculas})\right)\right)$$

Paso 3: la diferencia:

$$\text{Nolan} - \text{Inarritu}$$

nombre
Matthew McConaughey
Cillian Murphy

Consultas no monótonas

Con la diferencia podemos expresar:

- "Actores que **NO** aparecen en películas de SciFi"
- "Películas que **NO** tienen actores registrados"
- "Directores que **NO** han dirigido dramas"

Todas estas son consultas **no monótonas**: agregar datos puede hacer que el resultado disminuya.

Intersección ($\cap$)

Sean R_1, R_2 relaciones con los mismos atributos, su intersección $R_1 \cap R_2$ contiene las tuplas que están en **ambas** relaciones.

Ejemplo: Liste los actores dirigidos por C. Nolan **y** por A. Iñárritu (ambos directores):

$$\pi_{\text{nombre}}\left(\text{Actores} \bowtie \text{Actuo_en} \bowtie \sigma_{\text{director}=\text{"C. Nolan"}}(\text{Peliculas})\right) \cap$$
$$\pi_{\text{nombre}}\left(\text{Actores} \bowtie \text{Actuo_en} \bowtie \sigma_{\text{director}=\text{"A. Iñárritu"}}(\text{Peliculas})\right)$$

Resultado: `{Leonardo DiCaprio}`

Intersección es un operador derivado de los que ya tenemos.

Ejercicio: exprese $\cap$ con ($\pi, \sigma, \cup, -, \times, \bowtie$)

División

Un patrón de consulta frecuente (y difícil)

Estas consultas aparecen constantemente en sistemas reales:

- "Estudiantes que han aprobado **todos** los ramos obligatorios para egresar"
- "Proveedores que suministran **todas** las piezas necesarias para un proyecto"
- "Usuarios que han comprado **todos** los productos de un bundle"
- "Empleados que cumplen **todos** los requisitos de un cargo"

Todas tienen algo en común: preguntan por una **cuantificación universal** ("para todo").

Con los operadores que conocemos ($\pi, \sigma, \cup, -, \times, \bowtie$) **se puede resolver**, pero es sorprendentemente difícil de escribir correctamente. La **división** captura este patrón.

La división en la práctica

Importante: A diferencia de los operadores anteriores, los sistemas de bases de datos no implementan de forma nativa esto.

Vamos a ver patrones para expresar esta consulta con SQL, pero son **notoriamente difíciles** de escribir bien. Entender la división en álgebra relacional nos ayuda a entender **qué estámos haciendo** al escribir estas consultas en SQL.

División (÷)

Sean $R(A, B)$ y $S(B)$ relaciones. La **división** $R \div S$ retorna las tuplas de $\pi_A(R)$ que están asociadas en R con **todas** las tuplas de S.

Informalmente: "¿Qué valores de A aparecen combinados con TODOS los valores de B en S?"

División: Ejemplo

Consulta: Actores que han actuado en **todas** las películas de C. Nolan.

Paso 1: Obtenemos pares (id_actor, id_pelicula) de actores que actuaron en películas:

$$\rho_R\left(\pi_{\text{id_actor, id_pelicula}}(\text{Actuo_en})\right)$$

Paso 2: Obtenemos los ids de las películas de Nolan:

$$\rho_S\left(\pi_{\text{id}}\left(\sigma_{\text{director}=\text{"C. Nolan"}}(\text{Peliculas})\right)\right)$$

S = {1, 5, 6} (Interstellar, Oppenheimer, Inception)

Paso 3: Dividimos:

$$R \div S$$

Resultado: los actores cuyo id_actor aparece en R combinado con **todos** los ids {1, 5, 6}. En nuestros datos, ningún actor actuó en las tres películas de Nolan.

División como operador derivado

La división se puede expresar con proyección, producto cruz y diferencia:

$$R \div S = \pi_A(R) - \pi_A((\pi_A(R) \times S) - R)$$

¿Por qué funciona?

1. $\pi_A(R)$ — todos los candidatos posibles (todos los valores de A en R)
2. $\pi_A(R) \times S$ — todas las combinaciones posibles de llos candidatos con valores de S
3. $(\pi_A(R) \times S) - R$ — combinaciones que **faltan** en R
4. $\pi_A(\ldots)$ — candidatos a los que les **falta** al menos un valor de S
5. $\pi_A(R) - (\ldots)$ — candidatos a los que **no les falta nada** (¡esos son los que queremos!)

> Cuando veamos SQL, verán que el doble `NOT EXISTS` sigue exactamente esta misma lógica: "no existe un valor en S para el cual no exista la combinación en R."

Árboles de expresiones

Representación en árbol

Las expresiones de álgebra relacional se pueden representar como **árboles**:

- Las **hojas** son relaciones
- Los **nodos internos** son operadores
- Se evalúan de abajo hacia arriba

Ejemplo: `π_nombre(σ_{director="C. Nolan"}(peliculas))`

```
    π_nombre
        |
  σ_{director="C. Nolan"}
        |
   peliculas
```

Árbol para consulta compleja

Consulta: Nombre de actores que actuaron en películas de C. Nolan

```
π_{nombre}(actores ⋈ actuo_en ⋈ σ_{director="C. Nolan"}(peliculas))
```

```
                π_nombre
                   |
            ⋈_{id=id_actor}
               /        \
          actores    ⋈_{id_pelicula=id}
                       /             \
                  actuo_en    σ_{director="C. Nolan"}
                                      |
                                  peliculas
```

Dos árboles diferentes pueden producir el **mismo resultado**:

Árbol A: Hacer el join completo, luego filtrar

```
σ_{director="C. Nolan"}
         |
  ⋈_{id_pelicula=id}
     /            \
actuo_en       peliculas
```

Árbol B: Filtrar primero, luego hacer el join

```
  ⋈_{id_pelicula=id}
     /            \
actuo_en    σ_{director="C. Nolan"}
                  |
               peliculas
```

El **Árbol B** es más eficiente: filtra primero, reduciendo el tamaño de la tabla antes del join.

> Esta idea de "empujar selecciones hacia abajo" es la base de la **optimización de consultas** que veremos más adelante.

Algunas equivalencias útiles:

Conmutatividad del join:

```
R ⋈ S  =  S ⋈ R
```

Asociatividad del join:

```
(R ⋈ S) ⋈ T  =  R ⋈ (S ⋈ T)
```

Pushdown de selección:

```
σ_c(R ⋈ S) = σ_c(R) ⋈ S     (cuando c solo refiere a atributos de R)
```

Pushdown de proyección:

```
π_A(R ⋈ S) = π_A(π_{A∪B}(R) ⋈ S)    (donde B son los atributos de join)
```

> Estas reglas son la base de los **optimizadores de consultas** en los DBMS.

Resumen y referencia

Operadores fundamentales vs. derivados

Fundamentales

— no se pueden expresar con los demás:

Operador	Símbolo	Aridad
Proyección	π	unario
Selección	σ	unario
Unión	$\cup$	binario
Diferencia	$-$	binario
Producto cruz	$\times$	binario
Renombre	ρ	unario

Derivados

— se definen con los fundamentales:

Operador	Se define como
Join ($\bowtie_c$)	$\sigma_c(R \times S)$
Natural join ($\bowtie$)	$\pi\sigma(R \times S)$ sobre atributos compartidos
Intersección ($\cap$)	$R - (R - S)$
División ($\div$)	$\pi_A(R) - \pi_A((\pi_A(R) \times S) - R)$

Jerarquía de expresividad

```
{π, σ, ∪, ×, ρ}   →  solo consultas monótonas

        + (−)       →  consultas no monótonas (negación)
                       permite: ∩, ÷, antijoin
```

Los cinco operadores fundamentales {π, σ, ∪, −, ×} (más ρ) forman un **conjunto mínimo completo** del álgebra relacional.

¿Qué NO puede expresar el álgebra relacional?

- **Agregación:** no hay COUNT, SUM, AVG, MAX, MIN
- **Ordenamiento:** no hay ORDER BY
- **Clausura transitiva:** no se puede calcular "todos los ancestros" recursivamente
- **Recursión:** no hay consultas recursivas

Estas limitaciones motivan extensiones como `GROUP BY`, `ORDER BY`, y CTEs recursivos en SQL, que veremos más adelante.

Ejercicios

Ejercicio 1: Proyección + Selección

Usando el esquema `Peliculas/Actores/Actuo_en`, escribe en álgebra relacional:

> "Nombre y año de todas las películas de categoría 'Drama' con calificación mayor a 8.0"

Ejercicio 1: Solución

$$\pi_{\text{nombre, año}}\left(\sigma_{\text{categoria}=\text{"Drama"}\wedge\text{calificacion}>8.0}(\text{Peliculas})\right)$$

nombre	año
The Revenant	2015
Oppenheimer	2023

Ejercicio 2: Join + Proyección

> "Nombre de todos los actores que han actuado en películas dirigidas por C. Nolan"

Ejercicio 2: Solución

$$\pi_{\text{Actores.nombre}}(\text{Actores} \bowtie \text{Actores.id} = \text{Actuo_en.id_actor}\text{Actuo_en} \bowtie_{\text{Actuo_en.id_pelicula=Peliculas.id}} \sigma_{\text{director="C. Nolan"}}(\text{Peliculas}))$$

nombre
Matthew McConaughey
Leonardo DiCaprio
Cillian Murphy

Ejercicio 3: Diferencia

> "Nombre de todos los actores que NO han actuado en ninguna película de C. Nolan"

Ejercicio 3: Solución

$$\pi_{\text{nombre}}(\text{Actores}) - \pi_{\text{Actores.nombre}}\left(\text{Actores} \bowtie \text{Actuo_en} \bowtie \sigma_{\text{director}=\text{"C. Nolan"}}(\text{Peliculas})\right)$$

nombre
Timothée Chalamet
Jessica Chastain

Ejercicio 4: División

> "Nombre de los actores que han actuado en TODAS las películas de categoría 'SciFi'"

Ejercicio 4: Solución

Paso 1: pares (id_actor, id_pelicula) de todas las actuaciones:

$$\rho_R\left(\pi_{\text{id_actor, id_pelicula}}(\text{Actuo_en})\right)$$

Paso 2: ids de películas SciFi:

$$\rho_S\left(\pi_{\text{id}}\left(\sigma_{\text{categoria=”SciFi”}}(\text{Peliculas})\right)\right)$$

S = {1, 3, 6} (Interstellar, Dune, Inception)

Paso 3: división y nombre:

$$\pi_{\text{nombre}}\left(\text{Actores} \bowtie_{\text{actores.id=id_actor}} (R \div S)\right)$$

En nuestros datos: ningún actor actuó en las tres películas SciFi.

Ejercicio 5: Composición compleja

> "Nombre de los directores cuyas películas tienen un promedio de calificación superior al de las películas de A. Iñárritu"

Pista: El álgebra relacional **no tiene** operador de promedio. Esta consulta **no se puede expresar** en álgebra relacional.

Próxima clase

SQL: el lenguaje práctico construido sobre el álgebra relacional.

Cada consulta SQL que escribirán tiene una expresión equivalente en álgebra relacional. Haber aprendido álgebra relacional les permitirá **entender qué hace SQL por debajo**.